MongoDB Associate Developer with Java

Topics:

1. MongoDB Overview and The Document Model
2. CRUD
3. Indexes
4. Data Modeling
5. Tools and Tooling
6. Drivers

NOTE:
Practice the questions thoroughly and understand the question pattern to strengthen your concepts. Consistent practice is the key to mastering MongoDB with Java development. Keep learning and keep building!

Study Guide v1.0 | Good luck with your certification!

Terms & Conditions

- **Personal Use Only:** This material is for the exclusive use of the authorized individual.
- **No Redistribution:** Sharing, uploading, or distributing this PDF in any form is strictly prohibited.
- **Intellectual Property:** All content, including question patterns and explanations, is protected. Copying is not permitted.
- **Security Protocol:** Any attempt to bypass access limitations or replicate the content will lead to termination of access.
- **Access limit:** This document is configured for a maximum of **30 openings**. Please use them wisely.
- **Integrity:** By accessing this document, you agree to uphold the highest standards of academic integrity.

Preparation Objective:
The primary aim is to master the foundational knowledge required for the MongoDB Associate Developer certification within **20 days**. Commitment to daily practice is essential.

Practice Quiz:
A structured practice quiz will be conducted to evaluate progress. Instructions and timing will be provided shortly.

Study Guide v1.0 | Good luck with your certification!

MongoDB Overview and The Document Model

What is MongoDB?

- A database that stores data as documents (like a JSON object)
- No need to define table structure beforehand
- Great for modern apps that need flexibility

What is a Document?

- A single record in MongoDB; more flexible than SQL rows.
- Each document is a set of key-value pairs.

What is a Collection?

- A group of documents; like an SQL table but without a fixed schema.

Data Types

- **BOOLEAN:** true/false (Flags)
- **DOUBLE:** Decimal numbers (Price/Rating)
- **OBJECT:** Embedded document (Nested data)

> **The Most Important Rule:** Every document must have a unique "_id" field

Quick Mapping

SQL Table → MongoDB Collection | SQL Row → MongoDB Document | SQL Column → MongoDB Field

---

Q1

Question

A developer is creating a MongoDB collection to store information about employees. Each employee document is required to include the field, date of birth. What BSON type should the developer use for this field?

Options

A) Boolean
B) String
C) Date
D) Null

Correct

C

Explanation

Date type is used to store date of birth in MongoDB.

Q2

Question

Which TWO are valid MongoDB BSON types?

Options

1) Number
2) Boolean
3) Float
4) Map
5) Double

Correct

2, 5

Explanation

Boolean and Double are valid BSON types in MongoDB, while Number, Float, and Map are not official BSON types.

0318

Q3

Question

A developer is creating a MongoDB collection to store user profiles. Each user document needs to track whether the user has verified their email address. What BSON type should the developer use for this field?

Options

A) String
B) Double
C) Boolean
D) Object

Correct

C

Explanation

Boolean type is used to store true/false values, which is perfect for tracking verification status like email verification.

Q4

Question

A developer is creating a MongoDB collection to store product information. Each product document needs to store the price of the item, including decimals (e.g., 19.99). What BSON type should the developer use for this field?

Options

A) Int32
B) Boolean

0318

C) String
0318
D) Double

Correct

D

Explanation

The Double BSON type is used to store floating-point values and is appropriate for representing decimal numbers like prices.

Q5

Question

A developer is creating a MongoDB collection to store customer information. Each customer document needs to include a complete address containing street, city, and zip code as nested fields. What BSON type should the developer use for the address field?

Options

A) Array
B) Boolean
C) Object
D) String

Correct

C

Explanation

Object type (embedded document) is used to store nested data structures, allowing the address to contain multiple sub-fields like street, city, and zip code within a single field.

0318

(This page is left blank for future notes or questions)

0318

CRUD Operations in MongoDB

What is CRUD?

CRUD = Create, Read, Update, Delete

The 4 basic operations for working with data.

1. CREATE (Insert)

Add new documents to a collection.

Methods: `insertOne()`, `insertMany()`

```
db.employees.insertOne({
  name: "Sai",
  isActive: true,
  salary: 50000.00
})
```

2. READ (Query/Find)

Retrieve documents from a collection.

Methods: `find()`, `findOne()`

```
db.employees.find({ name: "Sai" })
```

3. UPDATE (Modify)

Change existing documents.

Methods: `updateOne()`, `updateMany()`, `replaceOne()`

```
db.employees.updateOne(
  { name: "Sai" },
  { $set: { salary: 60000.00 } }
)
```

0318

4. DELETE (Remove)

Remove documents from a collection.

Methods: `deleteOne()`, `deleteMany()`

```
db.employees.deleteOne({ name: "Sai" })
```

Quick Memory Trick:

- **C** - Create - Add new data
- **R** - Read - View/find data
- **U** - Update - Change data
- **D** - Delete - Remove data

Remember:

- `insertOne()` / `insertMany()` - CREATE
- `find()` / `findOne()` - READ
- `updateOne()` / `updateMany()` - UPDATE
- `deleteOne()` / `deleteMany()` - DELETE

CRUD Rules: Single vs Multiple

Operation	Single Document	Multiple Documents
CREATE	insertOne()	insertMany()
READ	findOne()	find()
UPDATE	updateOne()	updateMany()
DELETE	deleteOne()	deleteMany()

> **Rule:**
> "One" = Single document
> "Many" = Multiple documents

<

0318

0318

Q6

Question

Given the following sample documents in the inventory collection:

```
{ "_id": 6305, "name": "A.S.", "assignment": 5, "points": 24 }
{ "_id": 6308, "name": "B.M.", "assignment": 3, "points": 22 }
{ "_id": 6312, "name": "E.M.", "assignment": 5, "points": 30 }
{ "_id": 6315, "name": "E.M.", "assignment": 2, "points": 12 }
{ "_id": 6319, "name": "R.S.", "assignment": 2, "points": 14 }
{ "_id": 6322, "name": "A.S.", "assignment": 1, "points": 10 }
{ "_id": 6234, "name": "R.S.", "assignment": 1, "points": 10 }
{ "_id": 6235, "name": "A.S.", "assignment": 3, "points": 14 }
```

What will delete only the document which stores "E.M." lowest "points" and returns the deleted document?

Options

A. db.inventory.deleteOne({ "name" : "E.M." }, { sort : { "points" : 1 }})

B. db.inventory.findOneAndDelete({ "name" : "E.M." }, { sort : { "points" : 1 }})

C. db.inventory.findOneAndDelete({ "name" : "E.M." }, { sort : { "points" : -1 }})

D. db.inventory.deleteOne({ "name" : "E.M." }, { sort : { "points" : -1 }})

Correct

B. db.inventory.findOneAndDelete({ "name" : "E.M." }, { sort : { "points" : 1 }})

Explanation

Step by Step:

```
1. Find documents with name "E.M.":
   - { "_id": 6312, "name": "E.M.", "points": 30 }
   - { "_id": 6315, "name": "E.M.", "points": 12 }

2. Sort by points ascending (1 = lowest first):
   - { "_id": 6315, "points": 12 } (LOWEST)
   - { "_id": 6312, "points": 30 }

3. findOneAndDelete() deletes AND returns the document:
   - Deletes document with _id: 6315 (points: 12)
   - Returns the deleted document
```

0318

0318

Rules Summary

Operation	Purpose
findOneAndDelete()	Deletes AND returns the document
deleteOne()	Deletes only (does NOT return)
sort: { points: 1 }	1 = Ascending (lowest to highest)
sort: { points:-1 }	-1 = Descending (highest to lowest)

Key Takeaway

- To delete AND return - use findOneAndDelete()
- To delete only - use deleteOne()
- sort: 1 - lowest first
- sort: -1 - highest first

Q7

Question

Given the following sample documents in products collection:

```
{ "name": "XPhone", "price": 799, "color": ["white", "black"], "storage": [64, 128, 256] }
{ "name": "XPad", "price": 899, "color": ["white", "black", "purple"], "storage": [128, 256, 512] }
{ "name": "GTablet", "price": 899, "color": ["blue"], "storage": [16, 64, 128, 1024] }
{ "name": "GPad", "price": 699, "color": ["white", "orange", "gold", "gray"], "storage": [64, 128, 256] }
{ "name": "GPhone", "price": 599, "color": ["white", "orange", "gold", "gray"], "storage": [128, 256, 512] }
```

```
db.products.find({
  $and: [
    { "price": { $lte: 800 } },
    { $or: [ { "color": "black" }, { "storage": 1024 } ] }
  ]
})
```

0318

What is the correct output of the query?

0318

Options

1. { "name": "GTablet", "price": 899, "color": ["blue"], "storage": [16, 64, 128, 1024] }

2. { "name": "XPad", "price": 899, "color": ["white", "black", "purple"], "storage": [128, 256, 512] }
{ "name": "GTablet", "price": 899, "color": ["blue"], "storage": [16, 64, 128, 1024] }

3. { "name": "XPhone", "price": 799, "color": ["white", "black"], "storage": [64, 128, 256] }
{ "name": "GPad", "price": 699, "color": ["white", "orange", "gold", "gray"], "storage": [64, 128, 256] }

4. { "name": "GPhone", "price": 599, "color": ["white", "orange", "gold", "gray"], "storage": [128, 256, 512] }

Correct

3

Explanation

```
$lte - price <= 800
$or - either condition true (color = black OR storage = 1024)
$and - both conditions must be true

Step:
1. price <= 800 - XPhone, GPad, GPhone
2. (black OR 1024) - only XPhone has "black"
3. Final output - { "name": "XPhone", "price": 799, "color": ["white", "black"], "storage": [64, 128, 256] }
```

Key Takeaway

- $and means all conditions must be true - $or means any one condition is enough - $lte filters values less than or equal - Always apply filters step by step

Q8

Question

A collection named Fruit contains the following documents:

```
{ _id: 1, Fruit: "Orange", n: [1, 2, 3] }
{ _id: 3, Color: "Blue", Hobbie: "Cricket", x: 23 }
{ _id: 4, n: "Cat", Fruit: "Banana", x: 5.6 }
```

0318

```
{ _id: 7, Fruit: "Apple", Color: "Green", Price: 1.99 }
{ _id: 9, Fruit: "Pineapple", Color: "Red", Size: "Medium" }
```

The collection has a compound index on fields Fruit and Color.

Which document can be successfully added into the Fruit collection?

Options

1. { _id: 3, Fruit: "Apple", Type: "Fruit", Color: "Red" }

2. { _id: 8, Fruit: "Orange", n: [4, 5, 6] }

3. { _id: 5, Fruit: "Pineapple", Color: "Red", Size: "Large" }

4. { _id: 6, Fruit: "Banana", Color: "Yellow", Price: 2.99 }

Correct

4

Explanation

```
Compound Index - combination of (Fruit + Color) must be unique

Check:
Option 1 - _id already exists (3)
Option 2 - missing Color field
Option 3 - duplicate combination (Pineapple, Red)
Option 4 - new unique combination (Banana, Yellow)
```

Q9

Question

A developer is managing a collection named inventory. The collection contains the following documents:

```
{ _id: 1, item: "Bottle", specs: { brand: "Brandx", model: "X123" }, price:
550 }
{ _id: 2, item: "Textbook" }
{ _id: 3, item: "Notepad", author: "John Doe", stock: 120, specs: { brand:
"Brandy", model: "Y456" }, tags: ["new","hot"] }
```

Which TWO documents can be added into the inventory collection as new documents?

0318

Options

1. { _id: 4, item: "Tablet", specs: { brand: "NN", model: "Z999" }, price: 320 }
2. { _id: 2, item: "Book", author: "John Doe", stock: 50 }
3. { _id: 1, item: "Laptop", specs: { brand: "MM", model: "X123" }, price: 650 }
4. { _id: 5, item: "Notebook", author: "James", stock: 50 }
5. { _id: 3, item: "Phone", specs: { brand: "Brandy", model: "D56" }, tags: ["new","hot"] }

Correct

1 and 4

Explanation

```
_id - must be unique in a collection

Check:
Option 1 - new _id (4)
Option 2 - duplicate _id (2)
Option 3 - duplicate _id (1)
Option 4 - new _id (5)
Option 5 - duplicate _id (3)
```

Q10

Question

Given the following Java code:

```
Document document = new Document("name", "Pat")
.append("contact", new Document("phone", "555-555-555")
.append("email", "pat@mongodb.com"))
.append("location", Arrays.asList(-33.92502, 80.8279))
.append("Interests", Arrays.asList("Cooking", "Video Games", "Hiking"));
```

What Java code inserts this document?

Options

1. collection.insertone(document);

0318

2. document.insertone(collection);

3. document.insertOne(collection);

4. collection.insertOne(document);

Correct

4

Explanation

```
insertOne - method used on collection to insert a document
Syntax - collection.insertOne(document)
Other options wrong due to incorrect method name or wrong object usage
```

Q11

Question

A collection named cars contains the following documents:

```
{ _id: 1, model: "Highlander", maxSpeed: 150, Fueltype: "Petrol" }
{ _id: 2, model: "Rangerover", maxSpeed: 220, Fueltype: "Diesel" }
{ _id: 3, model: "Toyota", maxSpeed: 130, Fueltype: "CNG" }
{ _id: 4, model: "Suzuki", maxSpeed: 110, Fueltype: "Petrol" }
{ _id: 5, model: "Nissan", maxSpeed: 120, Fueltype: "Diesel" }
{ _id: 6, model: "Ford", maxSpeed: 135, Fueltype: "Diesel" }
{ _id: 7, model: "Hyundai", maxSpeed: 115, Fueltype: "CNG" }
{ _id: 8, model: "Tata", maxSpeed: 100, Fueltype: "Petrol" }
```

```
db.cars.find({
  $and: [
    { maxSpeed: { $gt: 115, $lte: 150 } },
    { $or: [ { Fueltype: "CNG" }, { Fueltype: "Diesel" } ] }
  ]
})
```

What is the expected output?

Options

1. { _id: 5, model: "Nissan", maxSpeed: 120, Fueltype: "Diesel" } { _id: 6, model: "Ford", maxSpeed: 135, Fueltype: "Diesel" }

2. { _id: 3, model: "Toyota", maxSpeed: 130, Fueltype: "CNG" } { _id: 5, model: "Nissan", maxSpeed: 120, Fueltype: "Diesel" } { _id: 6, model: "Ford", maxSpeed: 135, Fueltype: "Diesel" }

3. { _id: 3, model: "Toyota", maxSpeed: 130, Fueltype: "CNG" } { _id: 6, model: "Ford", maxSpeed: 135, Fueltype: "Diesel" }

Correct

2

Explanation

```
$gt - greater than 115
$lte - less than or equal to 150
$or - either CNG or Diesel
$and - both conditions must be true

Valid:
Toyota (130, CNG), Nissan (120, Diesel), Ford (135, Diesel)
```

Q12

Question

What Java statement finds and returns a single document?

Options

1. coll.find({"hello": "world"}).first();

2. coll.find(eq("hello", "world")).first();

3. coll.findone("hello", "world");

4. coll.findone(eq("hello", "world"));

Correct

2

Explanation

```
0318
find() - used to query documents
eq() - filter condition (field = value)
first() - returns only one document

Correct syntax - coll.find(eq("hello","world")).first()
Other options have wrong syntax or invalid method names
```

Q13

Question

Given a people collection with the following documents:

```
{ _id: 0, name: "Emma", sports: ["Soccer", "Badminton"] }
{ _id: 1, name: "James", sports: ["Basketball", "Tennis"] }
{ _id: 2, name: "Lucas" }
{ _id: 3, name: "Sophia", sports: "Tennis" }
{ _id: 4, name: "Mike", sports: ["Basketball"] }
{ _id: 5, name: "Grace", sports: ["Soccer", "Basketball", "Badminton", "Table Tennis"] }
```

```
db.people.find({ sports: "Tennis" })
```

What option lists all the documents that match the query?

Options

1. { _id: 1, name: "James", sports: ["Basketball", "Tennis"] } { _id: 3, name: "Sophia", sports: "Tennis" }

2. { _id: 1, name: "James", sports: ["Basketball", "Tennis"] }

3. { _id: 3, name: "Sophia", sports: "Tennis" } { _id: 5, name: "Grace", sports: ["Soccer", "Basketball", "Badminton", "Table Tennis"] }

Correct

1

Explanation

0318

```
0318
MongoDB matches both:
- exact value - "Tennis"
- array containing value - ["Basketball","Tennis"]

So matched:
James (array contains Tennis)
Sophia (exact Tennis)
```

Q14

Question

A collection named users contains the following documents:

```
{ _id: 1, name: "Alice", age: 30, active: true }
{ _id: 2, name: "Bob", age: 25, active: false }
{ _id: 3, name: "Charlie", age: 35, active: true }
{ _id: 4, name: "Kevin", age: 40, active: false }
```

```
db.users.find({
  $or: [
    { $and: [ { active: true }, { age: { $gt: 30 } } ] },
    { $and: [ { active: false }, { age: { $lt: 30 } } ] }
  ]
})
```

Which documents will be returned?

Options

1. { _id: 1, name: "Alice", age: 30, active: true } { _id: 3, name: "Charlie", age: 35, active: true }

2. { _id: 1, name: "Alice", age: 30, active: true } { _id: 2, name: "Bob", age: 25, active: false }

3. { _id: 2, name: "Bob", age: 25, active: false } { _id: 3, name: "Charlie", age: 35, active: true }

0318

```
{ _id: 3, name: "Charlie", age: 35, active: true }
```

Correct

3

Explanation

```
$gt - age > 30
$lt - age < 30
$and - both conditions must be true
$or - either condition group can match

Check:
active true & age > 30 - Charlie
active false & age < 30 - Bob

Result - Bob and Charlie
```

Q15

Question

The customer collection includes the following documents:

```
{
  "_id": 1,
  "customer": "Alice",
  "items": [
    { "product": "Pen", "qty": 5 },
    { "product": "Notebook", "qty": 3 }
  ]
}
{
  "_id": 2,
  "customer": "Bob",
  "items": [
    { "product": "Pencil", "qty": 2 },
    { "product": "Eraser", "qty": 1 }
  ]
}
{
  "_id": 3,
```

```
  "customer": "Charlie",
0318
  "items": [
    { "product": "Pen", "qty": 1 },
    { "product": "Ruler", "qty": 3 }
  ]
}
{
  "_id": 4,
  "customer": "Charlie",
  "items": [
    { "product": "Ruler", "qty": 1 },
    { "product": "Eraser", "qty": 5 }
  ]
}
```

```
db.customers.find({
  "items": {
    "$elemMatch": {
      "product": "Ruler",
      "qty": { "$gte": 3, "$lte": 4 }
    }
  }
})
```

What is the expected output?

Options

1. { "_id": 3, "customer": "Charlie", "items": [{ "product": "Pen", "qty": 1 }, { "product": "Ruler", "qty": 3 }] }

2. { "_id": 1, "customer": "Alice", "items": [{ "product": "Pen", "qty": 5 }, { "product": "Notebook", "qty": 3 }] }

3. { "_id": 4, "customer": "Charlie", "items": [{ "product": "Ruler", "qty": 1 }, { "product": "Eraser", "qty": 5 }] }

4. { "_id": 2, "customer": "Bob", "items": [{ "product": "Pencil", "qty": 2 }, { "product": "Eraser", "qty": 1 }] }

Correct

1

Explanation

0318

```
0318
$elemMatch - same array element must satisfy both conditions
$gte - qty >= 3
$lte - qty <= 4
```

Q16

Question

A collection coll has following documents:

```
{ _id: 4, type: "A", value: 60 }
{ _id: 5, type: "B", value: 80 }
{ _id: 6, type: "A", value: 30 }
```

A collection results has the following documents:

```
{ _id: 1, type: "A", value: 10 }
{ _id: 2, type: "B", value: 20 }
{ _id: 3, type: "C", value: 30 }
```

After executing the following aggregation pipeline:

```
db.coll.aggregate([
  { $match: { type: "A" } },
  { $out: "results" }
])
```

What is the result?

Options

1. Collection results has following documents: { _id: 1, type: "A", value: 10 } { _id: 2, type: "B", value: 20 } { _id: 3, type: "C", value: 30 } { _id: 4, type: "A", value: 60 } { _id: 5, type: "B", value: 80 } { _id: 6, type: "A", value: 30 }

2. Collection results has following documents: { _id: 1, type: "A", value: 10 } { _id: 2, type: "B", value: 20 } { _id: 3, type: "C", value: 30 } { _id: 4, type: "A", value: 60 } { _id: 6, type: "A", value: 30 }

0318

3. Collection results has following documents: { _id: 4, type: "A", value: 60 } { _id: 6, type: "A", value: 30 }

4. Collection results has following documents: { _id: 1, type: "A", value: 10 } { _id: 4, type: "A", value: 60 } { _id: 6, type: "A", value: 30 }

Correct

3

Explanation

```
$match → filters documents (type = "A")
$out → replaces entire target collection with result

So old data in "results" is removed and only matched documents are stored
```

Q17

Question

Given the following sample documents:

```
{ _id: 1, firstName: "Person1", company: "CompanyA", active: false }
{ _id: 2, firstName: "Person2", company: "CompanyB", active: false }
{ _id: 3, firstName: "Person3", company: "CompanyC", active: true }
{ _id: 4, firstName: "Person4", company: "CompanyD", active: true }
```

What code block should be used to delete the third document from a MongoDB collection in Java?

Options

```
1. collection.deleteOne(eq("_id", new ObjectId(3)));
```

```
2. collection.delete(eq("_id", new ObjectId(3)));
```

```
0318
3. collection.deleteOne(eq("_id", 3));
```

```
4. collection.delete(eq("_id", 3));
```

Correct

3

Explanation

```
deleteOne → deletes a single document
eq("_id", 3) → matches document with _id = 3
Other options use wrong method or incorrect ObjectId usage
```

Q18

Question

A MongoDB developer is working with the customers collection. The developer wants to return the username and email fields from each document.

Which TWO projection specifications meet this requirement?

Options

```
1.
db.customers.find({}, { username: 1, email: 1 })
```

```
2.
db.customers.find({}, { username: 1, email: 1, _id: 1 })
```

```
3.
db.customers.aggregate([
  { $project: { username: 1, email: 1, _id: 0 } }
])
```

```
4.                                                                    0318
```

```
db.customers.aggregate([
0318
  { $set: { username: 1, email: 1 } },
  { $unset: "_id" }
])
```

```
5.
db.customers.find({}, { username: 1, email: 1, _id: 0 })
```

Correct

3 and 5

Explanation

```
Projection rules:

- _id: 1 → _id field is INCLUDED (default behavior)
- _id not mentioned → still INCLUDED automatically
- _id: 0 → _id field is EXCLUDED

So:
Option 1 → returns username, email + _id (extra field)
Option 2 → explicitly includes _id
Option 3 → returns only username & email (correct)
Option 5 → returns only username & email (correct)
```

Q19

Question

Given the following sample documents:

```
{ _id: 1, company: "Farm Inc.", active: true }
{ _id: 2, company: "Garden Inc.", active: false }
```

A developer wants to create a search query to search for a company. There is a business requirement to accommodate misspellings. An Atlas Search index has already been created for this query.

What query satisfies these requirements?

Options

0318

```
0318
1.
db.users.aggregate([
  { $search: { text: { path: "company", query: "Gardin" } } }
])
```

```
2.
db.users.aggregate([
  { $search: { company: "Gardin", true } }
])
```

```
3.
db.users.aggregate([
  { $search: { text: { path: "company", query: "Gardin", fuzzy: {} } } }
])
```

```
4.
db.users.aggregate([
  { $search: { text: { path: "company", query: "Gardin", misspellings:
true } } }
])
```

Correct

3

Explanation

```
$search → used in Atlas Search
text → performs text search on field
fuzzy → allows matching even with spelling mistakes

So fuzzy search handles misspellings like "Gardin" → "Garden"
```

Q20

Question

Given the following match expression: 0318

0318

```
{ "c": { $eq: "Green" }, "a": { $gt: 3 } }
```

And the following documents:

```
{ _id: 1, p: 0.75, c: "Green", a: [1,3,5,7] }
{ _id: 2, p: 0.80, c: "Orange", a: [1,3] }
{ _id: 3, p: 0.13, c: ["Green","Orange"], a: [3] }
{ _id: 4, p: 0.00, c: "Green", a: [5,7] }
{ _id: 5, p: 0.04, c: "Green", a: 3 }
{ _id: 6, p: 0.97, c: "Red", a: 5 }
```

What document will be returned from this expression?

Options

```
1.
{ _id: 1, p: 0.75, c: "Green", a: [1,3,5,7] }
{ _id: 2, p: 0.80, c: "Orange", a: [1,3] }
{ _id: 4, p: 0.00, c: "Green", a: [5,7] }
```

```
2.
{ _id: 1, p: 0.75, c: "Green", a: [1,3,5,7] }
{ _id: 4, p: 0.00, c: "Green", a: [5,7] }
```

```
3.
{ _id: 1, p: 0.75, c: "Green", a: [1,3,5,7] }
{ _id: 3, p: 0.13, c: ["Green","Orange"], a: [3] }
{ _id: 4, p: 0.00, c: "Green", a: [5,7] }
```

```
4.
{ _id: 4, p: 0.00, c: "Green", a: [5,7] }
```

Correct

2

Explanation

0318

```
0318
$eq → c must be "Green"
$gt → a > 3

Array rule → if any element > 3, condition is satisfied

Match:
_id 1 → has 5,7
_id 4 → has 5,7

Others fail due to c not Green or a not > 3
```

Q21

Question

A developer needs to count the number of books published before 1997. The publish year is stored as field publishYear in the document.

Given the sample documents in the books collection:

```
{ _id: 1, name: "A concise history of Hong Kong", publishYear: 1996 }
{ _id: 2, name: "Music Theory and Practice", publishYear: 2008 }
{ _id: 3, name: "US Business Law: The Cases", publishYear: 1995 }
{ _id: 4, name: "Practical MongoDB Aggregations", publishYear: 2021 }
{ _id: 5, name: "Mastering MongoDB 7.0", publishYear: 1997 }
```

Which command should the developer use to achieve this goal?

Options

```
1.
db.books.find({ publishYear: { $lt: 1997 } }).toArray().count()
```

```
2.
db.books.countDocuments({ publishYear: { $lte: 1997 } })
```

```
3.
```

0318

```
db.books.countDocuments({ publishYear: { $lt: 1997 } })
0318
```

```
4.
db.books.countDocuments({ $lt: { publishYear: 1997 } })
```

Correct

3

Explanation

```
$lt → less than 1997 (before 1997)
countDocuments → counts matching documents

Option 3 correctly counts books published before 1997
```

Q22

Question

A MongoDB developer is managing a database that stores electricity meter readings for an energy provider.

The readings collection contains the following sample documents:

```
{ _id: 1, meterSerial: "MR0001", latestReading: 3753 }
{ _id: 2, meterSerial: "MR0002", latestReading: 1218 }
{ _id: 3, meterSerial: "MR0003", latestReading: 2105 }
{ _id: 4, meterSerial: "MR0004", latestReading: 2112 }
```

The developer plans to execute the following query in mongosh:

```
db.readings.find({}, { _id: 0, latestReading: 0 })
.sort({ latestReading: 1 })
.limit(2)
```

What outcome should the developer expect?

Options

0318

```
0318
1.
[
  { _id: 1, meterSerial: "MR0001", latestReading: 3753 },
  { _id: 4, meterSerial: "MR0004", latestReading: 2112 }
]
```

```
2.
[
  { meterSerial: "MR0002" },
  { meterSerial: "MR0003" }
]
```

```
3.
[
  { _id: 2, meterSerial: "MR0002", latestReading: 1218 },
  { _id: 3, meterSerial: "MR0003", latestReading: 2105 }
]
```

```
4.
[
  { meterSerial: "MR0001" },
  { meterSerial: "MR0004" }
]
```

Correct

2

Explanation

```
sort({ latestReading: 1 }) → ascending order → 1218, 2105, 2112, 3753
limit(2) → first 2 documents → MR0002, MR0003
projection → { _id: 0, latestReading: 0 } → only meterSerial shown
```

Q23

Question

0318

0318

A MongoDB developer is working on a collection called customers. The developer needs to identify documents where the accounts array contains at least one account number greater than 600000 and less than 1000000.

Which query should the developer use to achieve this?

Options

```
1.
db.customers.find({
  accounts: { $elemMatch: { $gt: 600000, $lt: 1000000 } }
})
```

```
2.
db.customers.find({
  accounts: { $all: [600000, 1000000] }
})
```

```
3.
db.customers.find({
  accounts: { $gt: 600000, $lt: 1000000 }
})
```

```
4.
db.customers.find({
  accounts: { $in: [600000, 1000000] }
})
```

Correct

1

Explanation

```
$elemMatch → ensures both conditions apply to the same array element
$gt → greater than 600000
$lt → less than 1000000

Other options either check exact values or apply conditions incorrectly
```

0318

0318

Q24

Question

An inventory database for a bookstore includes a collection named books. A developer needs to insert a new book record into the collection that includes the following fields, title: "Why MongoDB is Great" and location: "A5".

What command should the developer use to insert a new record into the books collection?

Options

```
1.
db.books.insertOne({ "title": "Why MongoDB is Great", "location": "A5" })
```

```
2.
db.books.insertOne("title": "Why MongoDB is Great", "location": "A5")
```

```
3.
db.books.insertOne({ title: "Why MongoDB is Great", location: A5 })
```

```
4.
db.books.insertOne({ title: "Why MongoDB is Great" location: "A5" })
```

Correct

1

Explanation

```
insertOne → inserts one document
Document must be in {} with key:value pairs
Values like "A5" must be in quotes
Other options have syntax errors
```

Q25

Question

0318

0318

A products collection needs to update the value of the qty field to 100 for a document with item "card". If a document with item "card" does not exist, a new document should be inserted.

What command should be used?

Options

```
1.
db.products.insertOne({ item: "card" }, { $set: { qty: 100 } }, { upsert:
true })
```

```
2.
db.products.updateOne({ item: "card" }, { $set: { qty: 100 } }, { upsert:
true })
```

```
3.
db.products.updateOne({ item: "card" }, { $set: { qty: 100 } })
```

```
4.
db.products.insertOne({ item: "card", qty: 100 }, { upsert: true })
```

Correct

2

Explanation

```
updateOne → updates a single document
$set → sets qty = 100
upsert: true → insert if document does not exist

So it handles both update and insert
```

Q26

Question

A collection named products contains 100 documents.

What method should be used to get all results from a cursor using mongosh?

0318

Options

0318

```
1.
db.products.findAll();
```

```
2.
db.products.find().toArray();
```

```
3.
db.products.find([]);
```

```
4.
db.products.forEach().toArray();
```

Correct

2

Explanation

```
find() → returns a cursor
toArray() → converts cursor into full result array

Other options are invalid methods or incorrect syntax
```

Q27

Question

An inventory database for a bookstore includes a collection named books. A developer needs to insert a new book record into the collection that includes the following fields, title: "Why MongoDB is Great" and location: "A5".

What command should the developer use to insert a new record into the books collection?

Options

```
1.
```

0318

```
db.books.insertOne({ title: "Why MongoDB is Great", location: "A5" })
0318
```

```
2.
db.books.insertOne("title": "Why MongoDB is Great", "location": "A5")
```

```
3.
db.books.insertOne({ title: "Why MongoDB is Great", location: A5 })
```

```
4.
db.books.insertOne({ title: "Why MongoDB is Great" location: "A5" })
```

Correct

1

Explanation

```
insertOne → inserts a document
Document must be inside {}
Values must be properly quoted
Other options have syntax errors
```

Q28

Question

Given the following documents in sales collection:

```
{ name: "Peter", year: 2023, item: "cars", currency: "US Dollar", amount: 5 }
{ name: "Peter", year: 2023, item: "cars", currency: "Euro", amount: 4 }
{ name: "Paul",  year: 2023, item: "cars", currency: "US Dollar", amount: 3 }
{ name: "Peter", year: 2022, item: "motorbikes", currency: "Euro", amount:
1 }
{ name: "Peter", year: 2023, item: "trucks", currency: "Euro", amount: 20 }
```

Given the following aggregation query:

0318

```
0318
db.sales.aggregate([
  { $match: { name: "Peter" } },
  { $group: { _id: { year: "$year", currency: "$currency" }, total: { $sum:
"$amount" } } }
])
```

What is the expected output?

Options

```
1.
[
  { _id: { year: 2023, currency: "US Dollar" }, total: 5 },
  { _id: { year: 2023, currency: "Euro" }, total: 24 },
  { _id: { year: 2022, currency: "Euro" }, total: 1 }
]
```

```
2.
[
  { _id: { year: 2023, currency: "US Dollar" }, total: 8 },
  { _id: { year: 2023, currency: "Euro" }, total: 24 }
]
```

```
3.
[
  { _id: { year: 2022, currency: "Euro" }, total: 1 },
  { _id: { year: 2023, currency: "Euro" }, total: 24 }
]
```

```
4.
[
  { _id: { year: 2023, currency: "US Dollar" }, total: 5 }
]
```

Correct

1

Explanation

0318

```
0318
$match → filters only name = "Peter"
$group → groups by year and currency
$sum → adds amounts

Peter records:
2023 US Dollar → 5
2023 Euro → 4 + 20 = 24
2022 Euro → 1
```

Q29

Question

A MongoDB developer needs to update multiple documents in the employee collection.

Which operation should the developer use to update the documents that match with a filter?

Options

```
1.
db.employee.updateMany({ a: "match" }, { $set: { b: "set" } })
```

```
2.
db.employee.updateMany({ $set: { b: "set" } }, { a: "match" })
```

```
3.
db.employee.findAndModify({ a: "match" }, { $set: { b: "set" } })
```

```
4.
db.employee.findAndUpdate({ a: "match" }, { $set: { b: "set" } })
```

Correct

1

Explanation

```
updateMany → updates multiple documents
0318
```

```
First argument → filter condition
0318
Second argument → update operation


Other options have wrong parameter order or invalid methods
```

Q30

Question

Given the following documents in a collection:

```
{ _id: 1, a: "one", b: "two" }
{ _id: 2, a: "two", b: "three" }
{ _id: 3, a: "one", b: "four", c: "three" }
```

Using the following update command:

```
db.coll.replaceOne({ a: "one" }, { a: "two", b: "five" })
```

What would the contents of the collection be after successfully running this command?

Options

```
1.
{ _id: 1, a: "two", b: "five" }
{ _id: 2, a: "two", b: "three" }
{ _id: 3, a: "one", b: "four", c: "three" }
```

```
2.
{ _id: 1, a: "one", b: "five" }
{ _id: 2, a: "two", b: "three" }
{ _id: 3, a: "one", b: "four", c: "three" }
```

```
3.
{ _id: 1, a: "two", b: "five" }
{ _id: 2, a: "two", b: "three" }
```

0318

```
{ _id: 3, a: "two", b: "five" }
```

0318

```
4.
{ _id: 1, a: "one", b: "two" }
{ _id: 2, a: "two", b: "three" }
{ _id: 3, a: "two", b: "five" }
```

Correct

1

Explanation

```
replaceOne → replaces only the first matching document
Filter → a = "one" → matches _id:1 first

Replacement → entire document replaced
So _id:1 becomes { a:"two", b:"five" }
Other documents remain unchanged
```

Q31

Question

An online gaming service wants to implement a policy that will ban players with three or more violations. A collection named players stores the number of violations for each player.

The players collection contains the following sample documents:

```
{ player_id: 1, violation: 0 }
{ player_id: 2, violation: 4 }
{ player_id: 3, violation: 4 }
```

Which command should be used to implement the three strike policy?

Options

```
1.
db.players.updateMany(
  { violation: { $gte: 3 } },
```

0318

```
  { $set: { Banned: true } }
0318
)
```

```
2.
db.players.updateMany(
  { violation: { $gt: 3 } },
  { $set: { Banned: true } }
)
```

```
3.
db.players.updateMany(
  { violation: { $gt: 3 } },
  { $setOnInsert: { Banned: true } },
  { upsert: false }
)
```

```
4.
db.players.updateMany(
  { violation: { $gt: 3 } },
  { $setOnInsert: { Banned: true } },
  { upsert: true }
)
```

Correct

1

Explanation

```
$gte → greater than or equal to 3 (three strike rule)
$set → adds/updates Banned field

Option 2 misses violation = 3
Option 3,4 use $setOnInsert (only for insert, not update)
```

Q32

Question

0318

Given the following documents in the orders collection:
0318

```
{ _id: 0, user: "C.A.", items: [ { item: "bag", size: 5 } ], status:
"shipped" }
{ _id: 1, user: "M.T.", items: [ { item: "polo", size: 10 } ], status: "order
placed" }
{ _id: 2, user: "S.R.", items: [ { item: "jacket", size: 6 } ], status: "in
progress" }
{ _id: 3, user: "T.C.", items: [ { item: "polo", size: 8 } ], status: "order
placed" }
{ _id: 4, user: "A.B.", items: [ { item: "bag", size: 7 } ], status: "new" }
{ _id: 5, user: "L.C.", items: [ { item: "hat", size: 5 } ], status:
"shipped" }
```

What mongosh command will find a document with status "order placed"?

Options

```
1.
db.orders.find_one({ status: "order placed" })
```

```
2.
db.orders.find_one(status: "order placed")
```

```
3.
db.orders.findOne({ status: "order placed" })
```

```
4.
db.orders.findOne(status: "order placed")
```

Correct

3

Explanation

```
findOne → returns a single matching document
Correct syntax → object inside {}
```

0318

```
Other options have wrong method name or incorrect syntax
0318
```

Q33

Question

A developer is working on a MongoDB database that contains a collection named customers. The developer needs to identify the documents where the accounts array contains "205563" or "721914".

Which query expression should be used to retrieve the documents?

Options

```
1.
db.customers.find({
  accounts: { $nin: [205563, 721914] }
})
```

```
2.
db.customers.find({
  accounts: { $all: [205563, 721914] }
})
```

```
3.
db.customers.find({
  accounts: { $elemMatch: { $eq: 205563, $eq: 721914 } }
})
```

```
4.
db.customers.find({
  accounts: { $in: [205563, 721914] }
})
```

Correct

4

Explanation

0318

```
0318
$in → matches if any value in array equals one of given values

So documents having either 205563 OR 721914 will be returned

Other options:
$nin → excludes values
$all → requires both values
$elemMatch → incorrect usage here
```

Q34

Question

The products collection has an Atlas Search index on the description field. A MongoDB developer needs to find all documents in the products collection where the description field contains the word "wireless".

Which query should the developer use to achieve this goal?

Options

```
1.
db.products.find({
  description: "wireless"
})
```

```
2.
db.products.aggregate([
  {
    $search: {
      text: {
        query: "wireless",
        path: "description"
      }
    }
  }
])
```

```
3.
db.products.find({
  $text: { $search: "wireless" }
0318
```

```
})
0318
```

```
4.
db.products.aggregate([
  {
    $match: {
      description: { $regex: "wireless" }
    }
  }
])
```

Correct

2

Explanation

```
Atlas Search → uses $search stage in aggregation
text → performs search on specified field
path → field name

Other options:
find → exact match
$text → requires text index, not Atlas Search
$regex → not using Atlas Search index
```

Q35

Question

Given a collection called coll that contains a single document:

```
{ _id: 1, a: 1 }
```

An update statement db.coll.updateOne({ a: 1 }, { $set: { "b.a": 1 } }) is successfully completed.

What is the output of the command db.coll.findOne() after the update statement?

Options

0318

```
0318
1.
{ _id: 1, a: 1, "b.a": 1 }
```

```
2.
{ _id: 1, a: 1, b: "a:1" }
```

```
3.
{ _id: 1, a: 1, b: { a: 1 } }
```

```
4.
{ _id: 1, a: 1 }
```

Correct

3

Explanation

```
"b.a" → dot notation creates nested document
So b becomes an object → { a: 1 }

Final structure → b: { a: 1 }
```

Q36

Question

A collection named Fruit contains the following documents:

```
{ _id: 1, Fruit: "Orange", n: [1, 2, 3] }
{ _id: 3, Color: "Blue", Hobbie: "Cricket", x: 23 }
{ _id: 4, n: "Cat", Fruit: "Banana", x: 5.6 }
{ _id: 7, Fruit: "Apple", Color: "Green", Price: 1.99 }
{ _id: 9, Fruit: "Pineapple", Color: "Red", Size: "Medium" }
```

0318

The collection has a unique index on fields Fruit and Color.
0318
Which document can be successfully added into the Fruit collection?

Options

```
1.
{ _id: 8, Fruit: "Orange", n: [4, 5, 6] }
```

```
2.
{ _id: 6, Fruit: "Banana", Color: "Yellow", Price: 2.99 }
```

```
3.
{ _id: 5, Fruit: "Pineapple", Color: "Red", Size: "Large" }
```

```
4.
{ _id: 3, Fruit: "Apple", Type: "Fruit", Color: "Red" }
```

Correct

2

Explanation

```
Unique index → combination of (Fruit + Color) must be unique

Check:
Option 1 → missing Color
Option 3 → duplicate (Pineapple, Red)
Option 4 → duplicate _id (3)
Option 2 → new combination (Banana, Yellow)
```

Q37

Question

Given the following sample documents in the inventory collection:

0318

0318

```
{ _id: 6301, item: "journal", qty: 25, size: { h: 14, w: 21, uom: "cm" } }
{ _id: 6302, item: "notebook", qty: 50, size: { h: 8.5, w: 11, uom: "in" } }
{ _id: 6303, item: "paper", qty: 100, size: { h: 8.5, w: 11, uom: "in" } }
{ _id: 6304, item: "planner", qty: 75, size: { h: 22.85, w: 30, uom: "cm" } }
{ _id: 6305, item: "postcard", qty: 45, size: { h: 10, w: 15.25, uom: "cm" } }
```

Which expression will remove only one document where size.h field is equal to 8.5?

Options

```
1.
db.inventory.deleteOne({ "size.h": 8.5 })
```

```
2.
db.inventory.deleteOne({ size.h: 8.5 })
```

```
3.
db.inventory.deleteOne({ size: { h: 8.5 } })
```

```
4.
db.inventory.deleteOne({ "size": { "h": { $eq: 8.5 } } })
```

Correct

1

Explanation

```
"size.h" → dot notation to access nested field
deleteOne → deletes only one matching document

Other options:
Option 2 → invalid syntax
Option 3 → requires exact match of full object
Option 4 → incorrect structure for nested field
```

0318

0318

Q38

Question

The students collection contains the following sample documents:

```
{
  student: { name: "Student A", class: "5A" },
  examResults: { english: [85, 90, 92], mathematics: [55, 70, 65] }
}
{
  student: { name: "Student B", class: "5A" },
  examResults: { english: [72, 70, 77], mathematics: [40, 55, 53] }
}
{
  student: { name: "Student C", class: "5B" },
  examResults: { english: [47, 48, 45], mathematics: [55, 62, 70] }
}
{
  student: { name: "Student D", class: "5A" },
  examResults: { english: [60, 62, 68], mathematics: [88, 91, 90] }
}
```

Given Query:

```
db.students.find({
  $and: [
    { "student.class": "5A" },
    {
      $and: [
        { "examResults.mathematics.0": { $lte: 60 } },
        { "examResults.mathematics.2": { $gte: 65 } }
      ]
    }
  ]
})
```

What is the expected result?

Options

```
1. The result set contains the documents of student A and C.
```

0318

```
0318
2. The result set contains the document of student A only.
```

```
3. The result set contains the document of student C only.
```

```
4. The result set contains the documents of student A, B and D.
```

Correct

2

Explanation

```
"student.class" = "5A" → A, B, D
mathematics[0] ≤ 60 and mathematics[2] ≥ 65

Check:
A → [55,70,65] → valid
B → [40,55,53] → fails (53 < 65)
D → [88,...] → fails (88 > 60)

Only Student A matches
```

Q39

Question

Given the following data set in the scores collection:

```
{ _id: ObjectId("512bc95fe835e68f199c8686"), author: "p1", score: 80 }
{ _id: ObjectId("512bc962e835e68f199c8687"), author: "p1", score: 85 }
{ _id: ObjectId("55f5a192d4bede9ac365b257"), author: "p2", score: 60 }
{ _id: ObjectId("55f5a192d4bede9ac365b258"), author: "p3", score: 55 }
{ _id: ObjectId("55f5a1d3d4bede9ac365b259"), author: "p4", score: 60 }
{ _id: ObjectId("55f5a1d3d4bede9ac365b25a"), author: "p3", score: 94 }
{ _id: ObjectId("55f5a1d3d4bede9ac365b25b"), author: "p5", score: 95 }
```

Given the following aggregation query:

0318

```
0318
db.scores.aggregate([
  { $match: { score: { $gt: 60 } } },
  { $group: { _id: "$author" } }
])
```

What is the output?

Options

```
1.
[ { _id: "p5" }, { _id: "p1" }, { _id: "p3" } ]
```

```
2.
[ { _id: "p1" }, { _id: "p1" }, { _id: "p3" } ]
```

```
3.
[ { _id: "p2" }, { _id: "p4" } ]
```

```
4.
[ { _id: "p1" }, { _id: "p2" }, { _id: "p3" }, { _id: "p4" }, { _id: "p5" } ]
```

Correct

1

Explanation

```
$match → score > 60 → p1, p1, p3, p5
$group → unique authors → p1, p3, p5
```

Q40

Question

Given the following documents in the HR collection:

0318

```
0318
{ "name": "B.M", "age": 36, "hobby": ["soccer", "baseball"] }
{ "name": "R.S", "age": 30, "hobby": ["soccer", "cycling", "baseball"] }
{ "name": "E.M", "age": 25, "hobby": ["cycling", "basketball"] }
```

What is the output of:

```
db.HR.find({ age: { $in: [25, 36] } }, { _id: 0 })
```

Options:

```
1.
{ "name": "B.M", "age": 36, "hobby": ["soccer", "baseball"] }
```

```
2.
{ "name": "R.S", "age": 30, "hobby": ["soccer", "cycling", "baseball"] }
```

```
3.
{ "name": "E.M", "age": 25, "hobby": ["cycling", "basketball"] }
{ "name": "R.S", "age": 30, "hobby": ["soccer", "cycling", "baseball"] }
{ "name": "B.M", "age": 36, "hobby": ["soccer", "baseball"] }
```

```
4.
{ "name": "B.M", "age": 36, "hobby": ["soccer", "baseball"] }
{ "name": "E.M", "age": 25, "hobby": ["cycling", "basketball"] }
```

Correct

4

Explanation

```
$in → matches any value in list [25, 36]
Projection { _id: 0 } → removes _id

Matched:
age 36 → B.M
```

0318

```
age 25 → E.M
0318
```

Q41

Question

Given the following documents in the incidents collection:

```
{ _id: 232, ts: ISODate("2022-05-20T12:02:00Z"), component: "ABC", status:
"Closed" }
{ _id: 153, ts: ISODate("2022-05-22T10:20:00Z"), component: "ABC", status: "In
Progress" }
{ _id: 353, ts: ISODate("2022-05-24T22:09:00Z"), component: "XYZ", status:
"Opened" }
{ _id: 464, ts: ISODate("2022-05-24T09:12:00Z"), component: "XYZ", status:
"Opened" }
{ _id: 212, ts: ISODate("2022-05-24T12:23:00Z"), component: "XYZ", status:
"Closed" }
{ _id: 484, ts: ISODate("2022-05-24T20:46:00Z"), component: "ABC", status: "In
Progress" }
```

What command updates only the earliest opened incident, changes its status to "In Progress", and returns the original document?

Options

```
1.
db.incidents.updateOne(
  { status: "Opened" },
  { $set: { status: "In Progress" } },
  { sort: { ts: 1 } }
)
```

```
2.
db.incidents.updateOne(
  { status: "Opened" },
  { $set: { status: "In Progress" } },
  { sort: { ts: -1 } }
)
```

0318

```
0318
3.
db.incidents.findOneAndUpdate(
  { status: "Opened" },
  { $set: { status: "In Progress" } },
  { sort: { ts: 1 } }
)
```

```
4.
db.incidents.findOneAndUpdate(
  { status: "Opened" },
  { $set: { status: "In Progress" } },
  { sort: { ts: -1 } }
)
```

Correct

3

Explanation

```
findOneAndUpdate → updates ONE and returns document
default → returns original document

sort: { ts: 1 } → earliest (smallest timestamp)

updateOne → does not return document
```

Q42

Question

The products collection contains a single text field called description. A MongoDB developer wants to create a text search index on the description field.

Which command should the developer use to achieve this?

Options

```
1.
db.products.createSearchIndex({
  name: "descIndex",
  definition: {
                                                    0318
```

```
    fields: {
      description: {
        type: "string"
      }
    }
  }
})
```

```
2.
db.products.createSearchIndex({
  name: "descIndex",
  definition: {
    mappings: {
      dynamic: true
    }
  }
})
```

```
3.
db.products.createIndex({ description: 1 })
```

```
4.
db.products.createIndex({ description: "text" })
```

Correct

4

Explanation

```
"text" index → used for text search in MongoDB

Option 1 & 2 → Atlas Search (different feature)
Option 3 → normal index (not for text search)
```

Q43

Question

0318

Given the following Python code:

0318

```
db = client.people
collection = db.employee

filter = { "position": "developer intern" }

update = {
  "$set": { "position": "developer regular" }
}

Assume client is a valid connection to MongoDB.
```

What is the correct syntax for updating multiple documents with a single command?

Options

```
1.
collection.update_multiple(update, filter)
```

```
2.
collection.update_many(update, filter)
```

```
3.
collection.update_multiple(filter, update)
```

```
4.
collection.update_many(filter, update)
```

Correct

4

Explanation

```
update_many(filter, update) → correct order

First → filter condition
Second → update operation
```

0318

```
0318
Other options:
wrong method name or wrong parameter order
```

Q44

Question

Consider a MongoDB collection that contains the following sample document:

```
{ "_id": "user123", "name": "John Doe", "age": 35, "status": "active" }
```

A db.collection.replaceOne() operation is executed with a new document:

```
{ "_id": "user123", "name": "Jane Smith", "age": 30 }
```

What is the expected state of the document after this operation?

Options

```
1.
{ "_id": "user123", "name": "Jane Smith", "age": 30 }
```

```
2.
{ "_id": "user123", "name": "John Doe", "age": 35, "status": "active" }
```

```
3.
{ "_id": "user123", "name": "Jane Smith", "status": "active" }
```

```
4.
{ "_id": "user123", "name": "Jane Smith", "age": 30, "status": "active" }
```

Correct

1

0318

Explanation

0318

```
replaceOne → replaces entire document (except _id)

Old fields like "status" are removed
Only new document fields remain
```

Q45

Question

Given the following sample from the metro_lines collection:

```
{
  line_name: "T1",
  route: ["Schofields", "Riverstone", "Vineyard"],
  preferred_line: ["red", "yellow", "green"]
}
{
  line_name: "T9",
  route: ["Riverstone", "Vineyard", "Mulgrave", "Schofields"],
  preferred_line: ["yellow", "green", "red"]
}
{
  line_name: "T5",
  route: ["East Richmond", "Vineyard", "Riverstone", "Schofields"],
  preferred_line: ["yellow", "red", "green"]
}
{
  line_name: "T11",
  route: ["Clarendon", "Windsor", "Mulgrave", "Riverstone"],
  preferred_line: ["green", "red", "yellow"]
}
```

What query should be used to find the most efficient route on the red line?

Options

```
1.
db.metro_lines.find({
  route: ["Riverstone", "Vineyard"],
  preferred_line: "red"
```

0318

```
})
0318
```

```
2.
db.metro_lines.find({
  route: { $all: ["Riverstone", "Vineyard"] },
  preferred_line: "red"
})
```

```
3.
db.metro_lines.find({
  route: { $in: ["Riverstone", "Vineyard"] },
  preferred_line: "red"
})
```

```
4.
db.metro_lines.find({
  route: ["Riverstone", "Vineyard"],
  preferred_line: { $in: ["red"] }
})
```

Correct

2

Explanation

```
$all → ensures BOTH "Riverstone" AND "Vineyard" exist in route
preferred_line: "red" → must include red

Other options:
array equality requires exact match
$in → matches any one value (not both)
```

Q46

Question

The books collection needs to update the value of the reorder field to true for all documents with a stock value less than 10.

0318

What command should be used in mongosh to achieve this?

0318

Options

```
1.
db.books.updateMany(
  { stock: { $lt: 10 } },
  { $set: { reorder: true } }
)
```

```
2.
db.books.update_many(
  { stock: { $lt: 10 } },
  { $set: { reorder: true } }
)
```

```
3.
db.books.update(
  { stock: { $lt: 10 } },
  { $set: { reorder: true } },
  { multi: true }
)
```

```
4.
db.books.updateOne(
  { stock: { $lt: 10 } },
  { $set: { reorder: true } },
  { multi: true }
)
```

Correct

1

Explanation

```
updateMany → updates all matching documents
$lt → stock < 10
$set → sets reorder = true

Option 3 is old syntax
```

0318

```
Option 4 updates only one document
0318
```

Q47

Question

Consider the following documents in the orders collection:

```
{ "orderId": 11, "status": "Pending", "amount": 120 }
{ "orderId": 12, "status": "Shipped", "amount": 528 }
{ "orderId": 13, "status": "Cancelled", "amount": 120 }
{ "orderId": 14, "status": "Shipped", "amount": 120 }
{ "orderId": 15, "status": "Pending", "amount": 300 }
```

Given the query:

```
db.orders.find(
  {
    status: { $in: ["Pending", "Shipped"] },
    amount: { $in: [120, 528] }
  },
  { _id: 0, orderId: 1, status: 1, amount: 1 }
)
```

What is the expected output?

Options

```
1.
[
  { "orderId": 12, "status": "Shipped", "amount": 528 },
  { "orderId": 14, "status": "Shipped", "amount": 120 }
]
```

```
2.
[
  { "orderId": 11, "status": "Pending", "amount": 120 },
  { "orderId": 12, "status": "Shipped", "amount": 528 },
  { "orderId": 14, "status": "Shipped", "amount": 120 }
0318
```

```
]
0318
```

```
3.
[
  { "orderId": 13, "status": "Cancelled", "amount": 120 }
]
```

```
4.
[
  { "orderId": 15, "status": "Pending", "amount": 300 }
]
```

Correct

2

Explanation

```
$in → matches any value in list

status → Pending or Shipped
amount → 120 or 528

Matching:
11 → Pending, 120
12 → Shipped, 528
14 → Shipped, 120

Others don't satisfy both conditions
```

Q48

Question

Given the following sample documents in the students collection:

```
{ _id: 0, name: "E.M.", class: "KL" }
{ _id: 1, name: "M.T.", class: "1H" }
                                                    0318
```

```
{ _id: 2, name: "S.F.", class: "2J" }
```
0318

What command is used to return only the names of all the students?

Options

```
1.
db.students.find({ name: 1 })
```

```
2.
db.students.find({ name: 1, _id: 0 })
```

```
3.
db.students.find(
{},
{ name: 1, _id: 0 }
)
```

```
4.
db.students.find(
{},
{ name: 1 }
)
```

Correct

3

Explanation

```
Projection → second parameter
{} → select all documents
{ name:1, _id:0 } → only name shown
```

Q49

Question

0318

0318

A MongoDB developer is working with the routes collection. The developer wants to find routes where the source airport is "KZN" OR the destination airport is either "EGO" or "LED".

What query should the developer use to meet this requirement?

Options

```
1.
db.routes.find({
$or: [
{ src_airport: "KZN" },
{ $and: [ { dst_airport: "EGO" }, { dst_airport: "LED" } ] }
]
})
```

```
2.
db.routes.find({
$and: [
{ src_airport: "KZN" },
{ $and: [ { dst_airport: "EGO" }, { dst_airport: "LED" } ] }
]
})
```

```
3.
db.routes.find({
$or: [
{ src_airport: "KZN" },
{
$or: [
{ dst_airport: "EGO" },
{ dst_airport: "LED" }
]
}
]
})
```

```
4.
db.routes.find({
$and: [
{ src_airport: "KZN" },
{ $or: [ { dst_airport: "EGO" }, { dst_airport: "LED" } ] }
]
```

0318

```
})
0318
```

Correct

3

Explanation

```
$or → any condition true
Need: src = KZN OR dst = EGO OR LED
Option 3 correctly applies OR logic
```

Q50

Question

Consider a users collection with the following documents:

```
{ _id: 1, name: "Alice", age: 28, city: "Paris" }
{ _id: 2, name: "Bob", age: 34, city: "Berlin" }
{ _id: 3, name: "Carol", age: 25, city: "Rome" }
```

A developer runs the following operation:

```
db.users.updateOne(
  { _id: 1 },
  { $set: { age: 29, city: "London" } }
)
```

What is the expected output of this operation?

Options

```
1.
{
  "acknowledged": false,
  "matchedCount": 1,
  "modifiedCount": 1,
  "upsertedId": null
```

0318

```
}
0318
```

```
2.
{
  "acknowledged": false,
  "matchedCount": 1,
  "modifiedCount": 1,
  "upsertedId": null
}
```

```
3.
{
  "acknowledged": true,
  "matchedCount": 1,
  "modifiedCount": 0,
  "upsertedId": { "$oid": "abc123" }
}
```

```
4.
{
  "acknowledged": true,
  "matchedCount": 1,
  "modifiedCount": 1,
  "upsertedId": null
}
```

Correct

4

Explanation

```
updateOne → updates one matching document

matchedCount = 1 → document found
modifiedCount = 1 → document updated
acknowledged = true → operation successful
upsertedId = null → no new document inserted
```

0318

Q51

Question

What returns the number of documents matching a given query?

Options

```
1.
db.coll.aggregate({ "$countDocuments": { "num": { "$lt": 5 } } })
```

```
2.
db.coll.countDocuments({ "num": { "$lt": 5 } })
```

```
3.
db.coll.estimatedDocumentCount({ "num": { "$lt": 5 } })
```

```
4.
db.coll.find({ "num": { "$lt": 5 } })
```

Correct

2

Explanation

```
countDocuments() → returns exact count with filter
estimatedDocumentCount() → ignores filter
find() → returns documents, not count
aggregate syntax in option 1 is incorrect
```

Q52

Question

The products collection includes the following sample document:

```
{
```

```
  _id: 100,
  quantity: 250,
  instock: true,
  reorder: false,
  details: { model: "14QQ", make: "Clothes Corp" },
  tags: ["apparel", "clothing"],
  ratings: [{ by: "Customer007", rating: 4 }]
}
```

```
db.products.updateOne(
  { _id: 100 },
  { $set: { "tags.1": "rain gear", "ratings.0.rating": 2 } }
)
```

What is the expected state of the document?

Options

1. { _id: 100, quantity: 250, instock: true, reorder: false, details: { model: "1400", make: "Clothes Corp" }, tags: ["rain gear", "clothing"], ratings: [{ by: "Customer007", rating: 2 }] }

2. { _id: 100, quantity: 250, instock: true, reorder: false, details: { model: "14QQ", make: "Clothes Corp" }, tags: ["apparel", "rain gear"], ratings: [{ by: "Customer007", rating: 6 }] }

3. { _id: 100, quantity: 250, instock: true, reorder: false, details: { model: "14QQ", make: "Clothes Corp" }, tags: ["apparel", "clothing", "rain gear"], ratings: [{ by: "Customer007", rating: 2 }] }

4. { _id: 100, quantity: 250, instock: true, reorder: false, details: { model: "14QQ", make: "Clothes Corp" }, tags: ["apparel", "rain gear"], ratings: [{ by: "Customer007", rating: 2 }] }

Correct

4

Explanation

```
$set - updates specific fields only
tags.1 - updates index 1 ("clothing" to "rain gear")
```

```
ratings.0.rating - updates first element rating (4 to 2)
```
0318

Key Takeaway

- $set updates only specified fields - Array index starts from 0 - tags.1 means second element - Nested fields can be updated using dot notation

0318

0318

MONGODB INDEX

Definition

Index is a data structure that helps MongoDB find data faster without scanning all documents (avoids COLLSCAN).

Key Points

```
1. find() on indexed field → FAST
2. insert/update on indexed field → SLOW (index also updates)
3. Index must match field name and order
4. For nested fields → use dot notation (a.b)
5. Direction (1 / -1) matters only for sorting
```

Important Concept

```
Without index → COLLSCAN → slow
With index → IXSCAN → fast
```

Sort Rule

```
Same order + same direction
OR full reverse direction
(Index supports both)
```

Order Rules (VERY IMPORTANT)

```
Best:
Equality → Sort → Range

Acceptable:
Equality → Range → Sort

Rules:
- Equality fields first
- Then sorting fields
- Then range conditions ($gt, $lt, etc.)
```

Performance Notes

```
Index speeds up read ↑
```

0318

```
Index slows down write ↓
Too many indexes → more memory + slower updates
```

Final Rule

> Correct field + correct order = best index performance All equality fields must come first

Q53

Question

Given the following query:

```
db.coll.find({}).sort({ timestamp: -1, status: 1 })
```

Which TWO indexes will improve the performance of this query the most? (Choose 2)

Options

```
1.
{ timestamp: 1, status: 1 }
```

```
2.
{ timestamp: -1, status: 1 }
```

```
3.
{ timestamp: 1, status: -1 }
```

```
4.
{ status: 1, timestamp: -1 }
```

```
5.
{ timestamp: -1, status: -1 }
```

Correct

0318 2 and 3

Explanation

```
Field order must match: timestamp → status

Direction rule: same direction or full reverse works

Query direction: (-1, 1)
Exact match index: (-1, 1)
Full reverse index: (1, -1)

Others fail due to wrong order or direction mismatch
```

Q54

Question

Given the following query that is performing a COLLSCAN (no suitable index is used):

```
db.test.find({}).sort({ name: 1, employeeID: 1 })
```

What index should the developer use to improve the performance of this query?

Options

```
1.
db.test.createIndex({ roll: 1, department: 1, name: 1, employeeID: 1 })
```

```
2.
db.test.createIndex({ _id: 1, name: 1, employeeID: 1 })
```

```
3.
db.test.createIndex({ employeeID: 1, name: 1 })
```

```
4.
```

0318

```
db.test.createIndex({ name: 1, employeeID: 1 })
```

0318

Correct

4

Explanation

```
Index must follow the same field order as the sort

Query sorts by: name → employeeID

So index should be: { name: 1, employeeID: 1 }

Extra fields before (Option 1, 2) break index usage
Wrong order (Option 3) cannot support this sort efficiently
```

Q55

Question

Given a sample collection called collection, in which all documents have the following structure:

```
{
_id: 1,
a: {
b: [1, 2, 3],
c: "example"
}
}
```

And the following query:

```
db.collection.find({ "a.b": 2 })
```

Which TWO indexes will improve the performance of the query?

Options

0318

0318

```
1. { "a.c": 1, "a.b": 1 }
```

```
2. { "a.b": 1 }
```

```
3. { a: 1, b: 1 }
```

```
4. { "a.b": 1, "a.c": 1 }
```

```
5. { a: 1 }
```

Correct

2 and 4

Explanation

```
Query uses field "a.b", so index must include "a.b"

{ "a.b": 1 } directly supports the query → BEST

Compound index works if "a.b" is the first field → { "a.b": 1, "a.c": 1 }

{ "a.c": 1, "a.b": 1 } is not efficient because "a.b" is not the first
field

{ a: 1 } and { a: 1, b: 1 } do not properly index nested field "a.b"
```

Q56

Question

A MongoDB developer is running the following query on the orders collection:

```
db.orders.find(
{ customerId: 123, status: "PENDING", orderDate: { $gte:
ISODate("2024-01-01") } }
```

0318

```
).sort({ orderDate: -1 })
```

0318

The query is performing a collection scan and is not using any existing indexes. Which index should the developer use to improve the query performance the MOST?

Options

```
1. { orderDate: -1, status: 1, customerId: 1 }
```

```
2. { customerId: 1, orderDate: -1 }
```

```
3. { customerId: 1, status: 1, orderDate: -1 }
```

```
4. { status: 1, orderDate: -1 }
```

Correct

3

Explanation

```
Follow rule: Equality → Equality → Range/Sort

customerId and status are equality filters → must come first

orderDate is used for range + sort → should come last

Index: { customerId: 1, status: 1, orderDate: -1 } supports filter + sort
efficiently

Other options either break order or miss fields → less efficient
```

Q57

Question

Given the following explain query:

0318

0318

```
db.foo.find({ txt: "hello" }).explain("queryPlanner")
```

Output (simplified):

```
{
explainVersion: "1",
queryPlanner: {
namespace: "test.foo",
parsedQuery: { txt: { $eq: "hello" } },
winningPlan: {
stage: "COLLSCAN",
filter: { txt: { $eq: "hello" } },
direction: "forward"
},
rejectedPlans: []
}
}
```

What statement is true for this explain plan?

Options

```
1. This is using an index.
```

```
2. This uses an index in reverse direction.
```

```
3. This has a sort stage with index support.
```

```
4. This is not using an index.
```

Correct

4

Explanation

```
stage: "COLLSCAN" → full collection scan
```

0318

```
No IXSCAN → no index used

MongoDB is scanning entire collection → slower performance
```

0318

Q58

Question

A collection of employees exists with the following structure:

```
db.test.find().pretty()

{
_id: ObjectId(...),
name: "Peter",
department: "it",
role: "software engineer",
employeeID: 1
}
{
_id: ObjectId(...),
name: "Katarzyna",
department: "hr",
role: "recruiter",
employeeID: 2
}
{
_id: ObjectId(...),
name: "Martina",
department: "sales",
role: "sales manager",
employeeID: 3
}
{
_id: ObjectId(...),
name: "Peter",
department: "sales",
role: "sales consultant",
employeeID: 4
}
```

A developer has been tasked with sorting employees based on fields: name, employeeID.

Given the following query that is performing a COLLSCAN:

0318

0318

```
db.test.find().sort({ name: 1, employeeID: 1 });
```

What index should the developer use to improve the performance of this query?

Options

```
1. db.test.createIndex({ roll: 1, department: 1, name: 1, employeeID: 1 });
```

```
2. db.test.createIndex({ _id: 1, name: 1, employeeID: 1 });
```

```
3. db.test.createIndex({ employeeID: 1, name: 1 });
```

```
4. db.test.createIndex({ name: 1, employeeID: 1 });
```

Correct

4

Explanation

```
Sort fields: name → employeeID

Index must follow same order

{ name: 1, employeeID: 1 } directly supports sorting

Extra fields before (Option 1, 2) break index usage

Wrong order (Option 3) cannot support this sort efficiently
```

Q59

Question

A MongoDB developer is working on a real estate listing application.
Given the following query pattern:

0318

0318

```
db.listings.find({
state: "Florida",
bedrooms: { $gt: 2 },
price: { $lte: 300000 }
}).sort({ build_in: -1 })
```

What index pattern will improve the performance for this query the MOST?

Options

```
1. { price: 1, bedrooms: 1, state: 1, build_in: 1 }
```

```
2. { build_in: 1, state: 1, price: 1, bedrooms: 1 }
```

```
3. { state: 1, build_in: 1, bedrooms: 1, price: 1 }
```

```
4. { bedrooms: 1, price: 1, state: 1, build_in: 1 }
```

Correct

3

Explanation

```
Rule: Equality → Sort → Range

state = equality → first

build_in = sort → second

bedrooms, price = range → last

Option 3 follows correct order

Others break order → less efficient
```

0318

0318

Q60

Question

A MongoDB developer is working on creating a database for a contact information application.

The database contains documents in the following format:

```
{
company: "ACME Inc.",
contacts: [
{
name: "John",
lastName: "Doe",
email: "[johndoe@email.com](mailto:johndoe@email.com)"
}
]
}
```

What index will maximize the read performance for queries that search for emails?

Options

```
1. { "contacts.email": 1 }
```

```
2. { "email.contacts": 1 }
```

```
3. { "contacts": 1 }
```

```
4. { "emails": 1 }
```

Correct

1

Explanation

```
Email is inside nested array field contacts
```

0318

```
Use dot notation → "contacts.email"

This creates a multikey index on emails

Other options use wrong field path or incorrect structure
```

0318

Q61

Question

Given the following schema sample in the inventory collection (with 1 million documents):

```
{ "_id": 1, "item": "Apple", "type": "fruit", "quantity": 500 }
```

What operations may become slower if the index { "item": 1, "quantity": 1 } is created on this collection?

Options

```
1.
db.inventory.find({ "item": "Apple" })
```

```
2.
db.inventory.updateMany({ "type": "food" }, { $set: { "status":
"Backorder" } })
```

```
3.
db.inventory.find({ "quantity": { $gte: 250, $lte: 400 } })
```

```
4.
db.inventory.updateMany({ "type": "fruit" }, { $inc: { "quantity": 100 } })
```

Correct

4

0318

0318

Explanation

```
Index improves read operations (find queries become faster)

But write operations become slower because index must also be updated

Updating indexed field "quantity" requires updating the index

Option 4 modifies "quantity" → indexed field → slower

Other operations either read or update non-indexed fields
```

Q62

Question

Given the following schema sample in the inventory collection (with 1 million documents):

```
{ "_id": 1, "item": "Apple", "type": "fruit", "quantity": 500 }
```

Which operation may become slower if the index { "item": 1, "quantity": 1 } is created on this collection?

Options

```
1.
db.inventory.find({ "item": "Apple" })
```

```
2.
db.inventory.updateMany({ "type": "fruit" }, { $inc: { "quantity": 100 } })
```

```
3.
db.inventory.updateMany({ "type": "food" }, { $set: { "status":
"Backorder" } })
```

```
4.
```

0318

```
db.inventory.find({ "quantity": { $gte: 250, $lte: 400 } })
```

0318

Correct

2

Explanation

```
Index makes reads faster but writes slower

Updating an indexed field requires updating the index

"quantity" is part of index → updates on it become slower

Option 2 modifies "quantity" → affected

Others are reads or update non-indexed fields
```

Q63

Question

Given the following query generates a collection scan:

```
db.grades.find({
class_id: 149,
"scores.score": { $gt: 75 }
}).sort({ "scores.type": 1 })
```

Which two indexes improve the performance of the query the most?

Options

```
1.
db.grades.createIndex({ "scores.type": 1, class_id: 1, "scores.score": 1 })
```

```
2.
db.grades.createIndex({ class_id: 1, "scores.type": 1, "scores.score": 1 })
```

```
3.
```

0318

```
db.grades.createIndex({ class_id: 1, "scores.score": 1, "scores.type": 1 })
```

0318

```
4.
db.grades.createIndex({ "scores.type": 1, "scores.score": 1, class_id: 1 })
```

Correct

2 and 3

Explanation

```
Rule: Equality → Sort → Range

class_id = equality → first

"scores.type" = sort → next

"scores.score" = range → last

Option 2 follows exact order → BEST

Option 3: range before sort → still usable but less optimal

Options 1 & 4 start with sort → cannot efficiently filter first
```

Q64

Question

What mongosh function displays the indexes associated with the pets collection?

Options

```
1. db.pets.showAllIndexes()
```

```
2. db.pets.indexesCollDetails()
```

```
3. db.pets.indexDetails()
```

0318

0318

```
4. db.pets.getIndexes()
```

Correct

4

Explanation

```
getIndexes() returns all indexes for a collection

Other options are invalid or non-existent functions
```

0318

0318

MongoDB Data Modeling

1. What is Data Modeling?

Designing how data is stored in MongoDB Decide: store together OR store separately Goal → Fast queries + less complexity

2. Embedding (Store Together)

Related data inside one document

```
{
  name: "John",
  orders: [
    { item: "Laptop", price: 50000 }
  ]
}
```

```
Use when:
- Data is always used together
- Small size
- One-to-few relationship

Advantage:
- Fast reads (no join needed)
```

3. Referencing (Store Separately)

Store data in different collections using references

```
Users:
{ _id: 1, name: "John" }

Orders:
{ userId: 1, item: "Laptop" }
```

```
Use when:
- Data is large
- Data is reused
- One-to-many or many-to-many

Advantage:
- Avoids duplication
```

0318

```
- Flexible updates
```

0318

4. Simple Rule to Remember

Embed → Together data Reference → Separate data

5. Modeling Idea

```
Think: "How will I query?"

If always together → embed
If separate usage → reference
```

NOTE

VERY IMPORTANT 2–3 questions will come If you understand → easy marks If you miss → easy loss Focus properly on this concept

Q65

Question

In an employee collection, each document represents an employee of the company. Which data model is the most effective for tracking the relationship between an employee and their manager?

Options

```
1. Reference array to employee documents in each manager document
```

```
2. Embed manager document into each employee document
```

```
3. Embed employee documents as an array into each manager document
```

```
4. Reference to manager document in each employee document
```

Correct

4

Explanation

0318

0318

```
Use referencing for hierarchical relationships

Each employee stores managerId → simple and scalable

Avoid embedding:
- Manager data may change
- Employees can be many → large document

So best model → reference manager in employee document
```

Q66

Question

There is a collection of patients for a medical practice. Which TWO attributes associated with the patients should be embedded in the patient document rather than referenced?

Choose 2 options.

Options

```
1. Visits to the practice
```

```
2. Their primary doctor
```

```
3. Current prescriptions
```

```
4. Diagnosed conditions
```

```
5. Their insurance provider
```

Correct

3 and 4

Explanation

0318

0318

```
Embed data that is closely related and frequently accessed together

Current prescriptions and diagnosed conditions → small, patient-specific
data

These are read together often → embedding is best

Visits → can grow large → better to reference

Primary doctor / insurance → shared across many patients → reference
```

Q67

Question

A MongoDB developer is creating a data model for a restaurant website that includes menu items and reviews.

The number of menu items is small, but the number of reviews can grow fast and is potentially unlimited.

How should the developer model the restaurant menu items and reviews?

Options

```
1. Embed both menu items and reviews into the restaurant document.
```

```
2. Embed menu items into the restaurant document, and reference to reviews
from the restaurant document.
```

```
3. Reference to both menu items and reviews from the restaurant document.
```

```
4. Reference to menu items from the restaurant document, and embed reviews
into the restaurant document.
```

Correct

2

Explanation

0318

```
0318
  Menu items → small and limited → embed

  Reviews → large and growing → reference

  Rule:
  Small + frequently used → Embed
  Large / unbounded → Reference
```

0318

0318

MongoDB Tools & Tooling

1. What is Tooling?

Tools = software used to work with MongoDB manually Not part of application code
Helps in managing, querying, and monitoring database

2. Common MongoDB Tools

```
mongosh
Command line tool
Used to run queries and commands

MongoDB Compass
GUI tool
Used to view data, run queries, create indexes visually

MongoDB Atlas
Cloud platform
Used to host, monitor, and manage databases

Atlas Search
Advanced search tool
Used for full-text search using $search
```

3. What Tools Do

```
Run queries
Create indexes
Monitor performance
View and edit data
Manage collections and databases
```

4. Important Concepts

```
Atlas Search
Uses $search stage (not normal find)
Requires search index
Supports fuzzy search (misspellings)

Explain Plan
Used to check performance
COLLSCAN → slow (no index)
IXSCAN → fast (index used)
```

0318

0318

```
Index Tools
db.collection.getIndexes()
Shows all indexes
```

5. Remember Rules (VERY IMPORTANT)

> Tools are used by developers directly Not used inside application code Atlas Search = Tool Compass = Tool mongosh = Tool

Q68

Question

A collection has the following documents:

```
{ _id: 1, lastName: "Doe", hobbies: "likes sports, reading, and bird watching" }
{ _id: 2, lastName: "Spears", hobbies: "likes to ride a bike and hiking" }
{ _id: 3, lastName: "Smith", hobbies: "likes to watch movies and medieval spears" }
```

A text search query follows the pattern: "find all people where 'spears' appears in any data field".

What Atlas Search index should be created?

Options

```
1. { "mappings": { "dynamic": true } }
```

```
2. { "mappings": { "default": true } }
```

```
3. { "mappings": { "dynamic": "runtime" } }
```

```
4. { "mappings": { "dynamic": "default" } }
```

Correct

0318

0318

1

Explanation

```
dynamic: true → indexes all fields automatically

Required when search is across any field

Other options are invalid or not applicable
```

Q69

Question

A collection has the following documents:

```
{ _id: 1, firstName: "John", hobbies: "likes sports, go to the movies, and
bird watching" }
{ _id: 2, firstName: "Jane", hobbies: "likes to ride a bike and hiking" }
{ _id: 3, firstName: "Alice", hobbies: "likes to watch movies and kite
flying" }
```

A text search query follows the pattern: "find all people whose hobbies are 'movies'". What Atlas Search index supports the requirements above?

Options

```
1.
{ "mappings": { "hobbies": "string" } }
```

```
2.
{ "mappings": { "dynamic": false, "properties": { "hobbies": { "type":
"string" } } } }
```

```
3.
{ "mappings": { "properties": { "hobbies": { "type": "string" } } } }
```

0318

0318

```
4.
{ "mappings": { "dynamic": false, "fields": { "hobbies": { "type":
"string" } } } }
```

Correct

4

Explanation

```
Atlas Search requires field definition for searchable fields

"hobbies" must be indexed as string

dynamic: false → only specified fields are indexed

"fields" section defines indexed fields

Other options:
Option 1 → invalid structure
Option 2 → uses "properties" (not correct for Atlas Search mappings)
Option 3 → missing dynamic control
```

Q70

Question

An Atlas Sample Dataset of restaurants is loaded into a database deployment.

How is Data Explorer used to find all restaurants whose name is "The Rose Palace" in the restaurants database?

Options

```
1.

* Select the collection
* Select "Aggregation" view
* Use $match: { name: "The Rose Palace" }
* Use $limit: 1
```

```
2.
```

0318

0318

```
* Select the collection
* Select "Find" view
* Enter filter: { name == "The Rose Palace" }
* Click "Find"
```

```
3.

MongoDB Data Explorer is primarily used to insert/edit/delete documents, not
find them
```

```
4.

* Select the collection
* Select "Find" view
* Enter filter: { name: "The Rose Palace" }
* Click "Find"
```

Correct

4

Explanation

```
Data Explorer → uses Find view + Filter for simple queries

Correct syntax → { name: "The Rose Palace" }

Aggregation not required for basic search

Option 2 wrong due to ==
Option 3 incorrect statement
```

Q71

Question

A developer will use the Data Explorer to find the most-viewed articles with an "active" status.

How should the developer configure the Data Explorer to achieve this?

0318

0318

Options

```
1.

Filter: { status: "active" }
Sort: { views: 1 }
Limit: 0
Skip: 0
```

```
2.

Filter: { status: "active" }
Sort: { views: -1 }
Limit: 1
Skip: 0
```

```
3.

Filter: { status: "active" }
Sort: { views: 1 }
Limit: 1
Skip: 0
```

Correct

2

Explanation

```
status: "active" → filters active articles

views: -1 → descending → highest views first

Limit: 1 → returns only the most-viewed article

Rule:
Most / highest → sort: -1
Lowest → sort: 1
```

0318

0318

Q72

Question

A developer is using Data Explorer in MongoDB Atlas to view a single document from the movies collection.

The developer has already opened the Atlas UI, selected the cluster, and navigated to the sample_mflix database.

What actions should the developer take next to view a single document?

Options

```
1.
Enter db.movies.find().limit(1) into the Filter field and click Find.
```

```
2.
Enter db.movies.findOne() into the Filter field and click Find.
```

```
3.
Enter { limit: 1 } into the Filter field and click Find.
```

```
4.
Leave the Filter field as {}, set Limit to 1, and click Find.
```

Correct

4

Explanation

```
Data Explorer uses Filter field for query conditions only

Commands like find() or findOne() are not used here

To get one document:
- Keep filter as {} (all documents)
- Set Limit = 1

Option 4 correctly follows UI usage
```

0318

0318

Q73

Question

What is the best practice to handle instances of MongoDB connection with the Python driver?

Options

```
1. A new MongoClient instance for each request
```

```
2. A pool of MongoClient instances to allow multiple concurrent database requests
```

```
3. A single, cached MongoClient instance
```

```
4. A new MongoClient instance for each logical transaction
```

Correct

3

Explanation

```
MongoClient is thread-safe and manages its own connection pool

Best practice → create one shared (cached) instance

Creating multiple instances increases overhead and reduces performance

Rule:
One MongoClient per application → best performance
```

Q74

Question

In MongoDB connection pooling, what is the purpose of maxPoolSize and minPoolSize?

0318

Options

0318

```
1.

maxPoolSize → minimum connections
minPoolSize → maximum connections
```

```
2.

maxPoolSize → maximum number of connections allowed in pool
minPoolSize → minimum number of connections maintained in pool
```

```
3.

maxPoolSize → number of databases
minPoolSize → number of collections
```

```
4.

maxPoolSize → query limit
minPoolSize → index limit
```

Correct

2

Explanation

```
maxPoolSize → limits total connections (controls load)

minPoolSize → keeps minimum ready connections (reduces delay)

Rule:
max → upper limit
min → always available connections
```

0318

0318

MongoDB Driver (Java)

1. What is a Driver?

Java Driver connects Java application to MongoDB Library → mongodb-driver-sync

```
MongoClient client = MongoClients.create("mongodb://localhost:27017");
MongoDatabase db = client.getDatabase("test");
```

2. Connecting to Atlas (IMPORTANT)

Use SRV connection string

```
MongoClient client = MongoClients.create(
"mongodb+srv://username:password@cluster0.mongodb.net/test"
);
```

```
SRV automatically:
- Finds cluster nodes
- Handles replica set
- No need to specify multiple hosts

Rule:
SRV → simplifies connection
TCP host list → not required manually
```

3. What Driver Does

```
Insert, update, delete, find

Executes queries from Java code

Handles connection internally
```

4. Connection Pooling

```
Java driver creates connection pool automatically

Reuses connections → better performance

Key Settings:
- maxPoolSize → max connections
```

0318

```
- minPoolSize → min ready connections
```

0318

5. Best Practice (VERY IMPORTANT)

Create ONE MongoClient Reuse it across application

6. Why Single Client?

```
MongoClient is thread-safe

Manages its own pool

Multiple clients → slow + heavy
```

7. Remember Rules

Driver = used in code MongoClient = entry point One MongoClient only SRV connection → preferred No need to manually manage TCP hosts Pooling handled automatically Rule Shortcut: One MongoClient + SRV → Best Practice

Q75

Question

What Java code is a valid aggregation projection that can be used with collection.aggregate(pipeline)?

Options

```
1.

Arrays.asList(
Aggregates.project(
Projections.fields(
Projections.include("role"),
Projections.computed(
"firstMaterial",
Document("$arrayElemAt", Arrays.asList("$materials", 0))
)
)
)
)
```

```
2.
```

0318

0318

```
Arrays.asList(
Aggregates.project(
Projections.fields(
Projections.include("role"),
Projections.computed(
"firstMaterial",
new Document("$arrayElemAt", Arrays.asList("$materials", 0))
)
)
)
)
```

```
3.

Arrays.asList(
Aggregates.project(
Projections.fields(
Projections.includeRole(),
Projections.computed(
"firstMaterial",
new Document("$arrayElemAt", Arrays.asList("$materials", 0))
)
)
)
)
```

```
4.

Arrays.asList(
Aggregates.project(
Projections.fields(
Projections.include("role"),
Projections.computed(
"firstMaterial",
new Document("$arrayElemAt", Arrays.asList("$materials", 1-4))
)
)
)
)
```

Correct

0318

0318 2

Explanation

```
Must use pipeline → Arrays.asList(...)

Must use Aggregates.project()

Projections.include("role") is correct syntax

Computed field must use new Document(...)

Key Rules:
Use new Document("$arrayElemAt", Arrays.asList("$materials", 0))

"$arrayElemAt" requires [array, index]

Index must be a single value (0, 1, 2...)

1-4 is invalid
includeRole() is invalid
Missing new Document is invalid

Rule Shortcut:
Pipeline + Aggregates + Projections + new Document = Correct
```

Q76

Question

The zip-entries collection contains documents with population information for cities within states. What Java method can be used to return a list of states whose total population across all cities is over 2,000,000?

Options

```
1.

private static List populationOverTwoMillion(MongoCollection zip_entries) {
Bson group = group("$state", sum("$pop"));
Bson match = match(gt("$pop", 2000000));

List results = zip_entries.aggregate(
Arrays.asList(group, match)
).into(new ArrayList());
```

0318

0318

```
return results;
}
```

```
2.

private static List populationOverTwoMillion(MongoCollection zip_entries) {
Bson group = group("$state", sum("totalPopulation", "$pop"));
Bson match = match(gt("totalPopulation", 2000000));

List results = zip_entries.aggregate(
Arrays.asList(group, match)
).into(new ArrayList());

return results;
}
```

Correct

2

Explanation

```
After grouping, MongoDB creates a new field to store the sum

You must name it → "totalPopulation"

Correct flow:
group("$state", sum("totalPopulation", "$pop"))

Now totalPopulation exists
Then match on totalPopulation

Why Option 1 is wrong:
sum("$pop") has no field name
match(gt("$pop", ...)) tries to filter original field

After grouping, original pop is not available

Rule Shortcut:
Always give name to computed field → then use it in match
```

0318

0318

Q77

Question

Which TWO statements are true about the Java driver?

Options

```
1. The driver's first authentication mechanism preference is PLAIN.
```

```
2. The driver automatically sets Read Preference to "primaryPreferred".
```

```
3. The driver uses JMX to allow for monitoring.
```

```
4. The driver can be setup via Maven or Gradle.
```

Correct

3, 4

Explanation

```
Java driver supports JMX → used for monitoring connection pool and
performance

Driver can be added using Maven or Gradle dependencies

Why others are wrong:
Default authentication is not PLAIN (it is SCRAM)
Default read preference is primary, not primaryPreferred

Rule Shortcut:
Java Driver → JMX + Maven/Gradle = Correct
```

Q78

Question

What syntax connects to a MongoDB Atlas cluster via the Java Driver using the DNS seed list connection format?

0318

0318

Options

```
1.

ConnectionString connectionString = new ConnectionString("mongodb://
cluster0.mongodb.net/test");
MongoClient mongoClient = MongoClient.create(connectionString);
```

```
2.

ConnectionString connectionString = new ConnectionString("mongodb://
_mongodb._tcp.cluster0.mongodb.net/test");
MongoClient mongoClient = MongoClient.create(connectionString);
```

```
3.

ConnectionString connectionString = new ConnectionString("mongodb+srv://
_mongodb._tcp.cluster0.mongodb.net/test");
MongoClient mongoClient = MongoClients.create(connectionString);
```

```
4.

ConnectionString connectionString = new ConnectionString("mongodb+srv://
cluster0.mongodb.net/test");
MongoClient mongoClient = MongoClients.create(connectionString);
```

Correct

4

Explanation

```
Atlas requires DNS seed list → must use mongodb+srv://

Do NOT use _mongodb._tcp → not valid in Java driver

Must use correct method → MongoClients.create()

Correct format:
mongodb+srv://cluster0.mongodb.net/test
```

0318

0318

```
Why others are wrong:
Option 1 → uses mongodb:// (not DNS format)
Option 2 → uses _mongodb._tcp (invalid)
Option 3 → mixes SRV with _tcp (wrong format)

Rule Shortcut:
mongodb+srv + MongoClients.create = Correct
```

0318

0318

Thank You

You have successfully completed all MongoDB concepts and practice questions.

Stay confident, revise well, and trust your preparation.

Good Luck for Your Certification!

0318